

使用 CREMA 根據 Pub/Sub 佇列量自動調度 Cloud Run worker 集區

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/crema-autoscaler-pubsub-workerpool?hl=zh-tw>

步驟數：12

瞭解如何使用 CREMA，根據 Pub/Sub 佇列容量自動調整 Cloud Run 工作站集區的大小

目錄

1. 簡介
2. 設定環境變數
3. 設定 Pub/Sub
4. IAM 與服務帳戶
5. 確認 SA 權限
6. 建構及部署消費者工作站集區
7. 設定 CREMA
8. 部署 CREMA 服務
9. 工作負載測試
10. 疑難排解
11. 恭喜！
12. 清除所用資源

步驟 1 · 約 0 分鐘

1. 簡介

總覽

本教學課程說明如何部署 Cloud Run 工作站集區 (消費者)，處理 Pub/Sub 訊息，並使用 [Cloud Run 外部指標自動調整資源配置 \(CREMA\)](#)，根據佇列深度自動調整消費者執行個體。

課程內容

本程式碼研究室涵蓋下列內容：

- 建立 Pub/Sub 主題和訂閱項目，並將訊息推送至該主題。
 - 部署 Cloud Run worker 集區 (消費者)，從 Pub/Sub 取用訊息。
 - 將 GitHub 上的 CREMA 專案部署為 Cloud Run 服務，根據 Pub/Sub 訂閱項目中的訊息數量，自動調整工作人員集區的大小。
 - 在本機執行 Python 指令碼產生負載，測試自動調度資源設定。
-

步驟 2 · 約 0 分鐘

2. 設定環境變數

由於本程式碼研究室會使用許多環境變數，建議您執行

```
set -u
```

如果您嘗試使用尚未設定的環境變數，系統會發出警告。如要還原這項設定，請執行 `set +u`

首先，請將下列變數改為您的專案 ID。

```
export PROJECT_ID=<YOUR_PROJECT_ID>
```

然後將其設為本程式碼研究室的專案。

```
gcloud config set project $PROJECT_ID
```

接著，請設定本程式碼研究室使用的環境變數。

```
export REGION=us-central1
export TOPIC_ID=crema-pubsub-topic
export SUBSCRIPTION_ID=crema-pubsub-sub
export CREMA_SA_NAME=crema-service-account
export CONSUMER_SA_NAME=consumer-service-account
export CONSUMER_WORKER_POOL_NAME=worker-pool-consumer
export CREMA_SERVICE_NAME=my-crema-service
```

為這個程式碼研究室建立目錄

□□□

```
mkdir crema-pubsub-codelab
cd crema-pubsub-codelab
```

啟用 API

```
gcloud services enable \
  artifactregistry.googleapis.com \
  cloudbuild.googleapis.com \
  run.googleapis.com \
  parametermanager.googleapis.com
```

最後，請確認 gcloud 使用的是最新版本

```
gcloud components update
```

步驟 3 · 約 0 分鐘

3. 設定 Pub/Sub

建立主題和提取訂閱項目，供工作站集區處理。Bash

建立主題。

```
gcloud pubsub topics create $TOPIC_ID
```

建立訂閱項目。

```
gcloud pubsub subscriptions create $SUBSCRIPTION_ID --topic=$TOPIC_ID
```

步驟 4 · 約 0 分鐘

4. IAM 與服務帳戶

建議您為每個 Cloud Run 資源建立服務帳戶。在本程式碼研究室中，您將建立下列項目：

- 消費者 SA：處理 Pub/Sub 訊息的 worker 集區身分。
- CREMA SA：CREMA 自動調整規模服務的身分。

建立服務帳戶

建立工作站集區消費者 SA：

```
gcloud iam service-accounts create $CONSUMER_SA_NAME \
  --display-name="PubSub Consumer Service Account"
```

建立工作站集區 CREMA 服務帳戶：

```
gcloud iam service-accounts create $CREMA_SA_NAME \
  --display-name="CREMA Autoscaler Service Account"
```

授予消費者 SA 權限

授予工作站集區消費者服務帳戶從訂閱項目提取訊息的權限。

```
gcloud pubsub subscriptions add-iam-policy-binding $SUBSCRIPTION_ID \
  --member="serviceAccount:$CONSUMER_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/pubsub.subscriber"
```

授予 CREMA SA 權限

CREMA 需要權限才能讀取參數、調整工作站集區大小，以及監控 Pub/Sub 指標。

1. 存取 Parameter Manager (設定讀取者)：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/parametermanager.parameterViewer"
```

2. 調度 Worker 集區資源 (Cloud Run 開發人員)：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \
  --role="roles/run.developer"
```

3. 監控 Pub/Sub：

授予監控檢視者角色。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
--member="serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \  
--role="roles/monitoring.viewer"
```

為訂閱項目新增政策，讓 CREMA 服務 SA 查看

```
gcloud pubsub subscriptions add-iam-policy-binding $SUBSCRIPTION_ID \  
--member="serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \  
--role="roles/pubsub.viewer"
```

CREMA SA 也需要「服務帳戶使用者」角色，才能變更執行個體數量：

```
gcloud iam service-accounts add-iam-policy-binding \  
$CONSUMER_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com \  
--member="serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \  
--role="roles/iam.serviceAccountUser"
```

步驟 5 · 約 0 分鐘

5. 確認 SA 權限

繼續進程式碼研究室之前，請確認 CREMA 服務帳戶具有正確的專案層級角色。

```
gcloud projects get-iam-policy $PROJECT_ID \
  --flatten="bindings[].members" \
  --format="table(bindings.role)" \
  --
  filter="bindings.members:serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com"
```

應會產生下列結果：

```
roles/monitoring.viewer
roles/parametermanager.parameterViewer
roles/run.developer
```

確認 Pub/Sub 訂閱項目具有允許 CREMA 服務 SA 查看的政策。

```
gcloud pubsub subscriptions get-iam-policy $SUBSCRIPTION_ID \
  --flatten="bindings[].members" \
  --
  filter="bindings.members:serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \
  --format="table(bindings.role)"
```

應會產生

```
roles/pubsub.viewer
```

並確認 CREMA SA 具有服務帳戶使用者角色

```
gcloud iam service-accounts get-iam-policy \
  $CONSUMER_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com \
  --flatten="bindings[].members" \
  --
  filter="bindings.members:serviceAccount:$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com"
```

應會產生下列結果

```
bindings:
  members: serviceAccount:crema-service-account@<PROJECT_ID>.iam.gserviceaccount.com
  role: roles/iam.serviceAccountUser
```

Worker Pool Consumer SA 具有 Pub/Sub 訂閱者角色

```
gcloud pubsub subscriptions get-iam-policy $SUBSCRIPTION_ID \
  --flatten="bindings[ ].members" \
  --
filter="bindings.members:serviceAccount:$CONSUMER_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com
" \
  --format="table(bindings.role)"
```

應會產生

```
ROLE
roles/pubsub.subscriber
```

6. 建構及部署消費者工作站集區

為消費者程式碼建立目錄並輸入。

```
mkdir consumer
cd consumer
```

1. 建立 `consumer.py` 檔案

```
import os
import time
from google.cloud import pubsub_v1
from concurrent.futures import TimeoutError

# Configuration
PROJECT_ID = os.environ.get('PROJECT_ID')
SUBSCRIPTION_ID = os.environ.get('SUBSCRIPTION_ID')

subscription_path = f"projects/{PROJECT_ID}/subscriptions/{SUBSCRIPTION_ID}"

print(f"Worker Pool instance starting. Watching {subscription_path}...")

subscriber = pubsub_v1.SubscriberClient()

def callback(message):
    try:
        data = message.data.decode("utf-8")
        print(f"Processing job: {data}")
        time.sleep(5) # Simulate work
        print(f"Done {data}")
        message.ack()
    except Exception as e:
        print(f"Error processing message: {e}")
        message.nack()

streaming_pull_future = subscriber.subscribe(subscription_path, callback=callback)
print(f"Listening for messages on {subscription_path}...")

# Wrap subscriber in a 'with' block to automatically call close() when done.
with subscriber:
    try:
        # When `timeout` is not set, result() will block indefinitely,
        # unless an exception is encountered first.
        streaming_pull_future.result()
    except TimeoutError:
        streaming_pull_future.cancel() # Trigger the shutdown.
        streaming_pull_future.result() # Block until the shutdown is complete.
    except Exception as e:
        print(f"Streaming pull failed: {e}")
```

2. 建立 Dockerfile

```
FROM python:3.12-slim
RUN pip install google-cloud-pubsub
COPY consumer.py .
CMD ["python", "-u", "consumer.py"]
```

3. 部署消費者工作站集區

本程式碼研究室建議先部署 0 個執行個體的 worker 集區，這樣您就能在 CREMA 偵測到訂閱項目中的 Pub/Sub 訊息時，觀察 CREMA 如何調度 worker 集區。

```
gcloud beta run worker-pools deploy $CONSUMER_WORKER_POOL_NAME \
  --source . \
  --region $REGION \
  --service-account="$CONSUMER_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" \
  --instances=0 \
  --set-env-vars PROJECT_ID=$PROJECT_ID,SUBSCRIPTION_ID=$SUBSCRIPTION_ID
```

步驟 7 · 約 0 分鐘

7. 設定 CREMA

1. 返回專案的根目錄。

```
cd ..
```

2. 建立設定檔建立名為 `crema-config.yaml` 的檔案

```
apiVersion: crema/v1
kind: CremaConfig
spec:
  pollingInterval: 30
  triggerAuthentications:
    - metadata:
        name: adc-trigger-auth
      spec:
        podIdentity:
          provider: gcp
  scaledObjects:
    - spec:
        scaleTargetRef:
          name:
            projects/PROJECT_ID_PLACEHOLDER/locations/REGION_PLACEHOLDER/workerpools/CONSUMER_WORKER_POOL_NAME_PLACEHOLDER
        triggers:
          - type: gcp-pubsub
            metadata:
              subscriptionName: "SUBSCRIPTION_ID_PLACEHOLDER"
              # Target number of undelivered messages per worker instance
              value: "10"
              mode: "SubscriptionSize"
            authenticationRef:
              name: adc-trigger-auth
```

3. 替代變數

```
sed -i "s/PROJECT_ID_PLACEHOLDER/$PROJECT_ID/g" crema-config.yaml
sed -i "s/REGION_PLACEHOLDER/$REGION/g" crema-config.yaml
sed -i "s/CONSUMER_WORKER_POOL_NAME_PLACEHOLDER/$CONSUMER_WORKER_POOL_NAME/g" crema-config.yaml
sed -i "s/SUBSCRIPTION_ID_PLACEHOLDER/$SUBSCRIPTION_ID/g" crema-config.yaml
```

4. 確認「`crema-config.yaml`」正確無誤

```
if grep -q "_PLACEHOLDER" crema-config.yaml; then
  echo "❌ ERROR: Validations failed. '_PLACEHOLDER' was found in crema-config.yaml."
  echo "Please check your environment variables and run the 'sed' commands again."
else
  echo "✅ Config check passed: No placeholders found."
fi
```

5. 上傳至 Parameter Manager

為 Parameter Manager 設定其他環境變數

```
export PARAMETER_ID=crema-config
export PARAMETER_REGION=global
export PARAMETER_VERSION=1
```

建立 Parameter 資源

```
gcloud parametermanager parameters create $PARAMETER_ID \
  --location=$PARAMETER_REGION \
  --parameter-format=YAML
```

建立參數版本 1

```
gcloud parametermanager parameters versions create $PARAMETER_VERSION \
  --parameter=crema-config \
  --project=$PROJECT_ID \
  --location=$PARAMETER_REGION \
  --payload-data-from-file=crema-config.yaml
```

確認參數已成功新增

```
gcloud parametermanager parameters versions list \
  --parameter=$PARAMETER_ID \
  --location=$PARAMETER_REGION
```

您應該會看到類似下列內容：

```
projects/<YOUR_PROJECT_ID>/locations/global/parameters/crema-config/versions/1
```

步驟 8 · 約 0 分鐘

8. 部署 CREMA 服務

在本節中，您將部署 CREMA 自動調整服務。您將使用公開的圖片。

1. 設定 CREMA 必要的環境變數

```
CREMA_CONFIG_PARAM_VERSION=projects/$PROJECT_ID/locations/$PARAMETER_REGION/parameters/$PARAMETER_ID/versions/$PARAMETER_VERSION
```

2. 驗證版本名稱路徑

```
echo $CREMA_CONFIG_PARAM_VERSION
```

內容應如下所示：

```
projects/<YOUR_PROJECT>/locations/global/parameters/crema-config/versions/1
```

3. 設定 CREMA 映像檔的環境變數

```
IMAGE=us-central1-docker.pkg.dev/cloud-run-oss-images/crema-v1/autoscaler:1.0
```

4. 並部署 CREMA 服務

請注意，您必須提供基本映像檔。

```
gcloud beta run deploy $CREMA_SERVICE_NAME \
  --image=$IMAGE \
  --region=${REGION} \
  --service-account="${CREMA_SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com" \
  --no-allow-unauthenticated \
  --no-cpu-throttling \
  --labels=created-by=crema \
  --base-image=us-central1-docker.pkg.dev/serverless-runtimes/google-24/runtimes/java25 \
  --set-env-
vars="CREMA_CONFIG=${CREMA_CONFIG_PARAM_VERSION},OUTPUT_SCALER_METRICS=True,ENABLE_CLOUD_LOGGING=True"
```

步驟 9 · 約 0 分鐘

9. 工作負載測試

1. 建立可將訊息發布至 Pub/Sub 主題的指令碼

```
touch load-pubsub.sh
```

2. 將下列程式碼新增至 `load-pubsub.sh` 檔案

```
#!/bin/bash
TOPIC_ID=${TOPIC_ID}
PROJECT_ID=${PROJECT_ID}
NUM_MESSAGES=100

echo "Publishing $NUM_MESSAGES messages to topic $TOPIC_ID..."

for i in $(seq 1 $NUM_MESSAGES); do
  gcloud pubsub topics publish $TOPIC_ID --message="job-$i" --project=$PROJECT_ID &
  if (( $i % 10 == 0 )); then
    wait
    echo "Published $i messages..."
  fi
done
wait
echo "Done. All messages published."
```

2. 執行負載測試

```
chmod +x load-pubsub.sh
./load-pubsub.sh
```

3. 監控縮放作業，等待 3 到 4 分鐘。查看 CREMA 記錄，瞭解系統如何根據新的 authenticationRef 設定建議執行個體。

```
gcloud logging read "resource.type=cloud_run_revision AND
resource.labels.service_name=$CREMA_SERVICE_NAME AND textPayload:SCALER" \
  --limit=20 \
  --format="value(textPayload)" \
  --freshness=5m
```

4. 監控處理程序：查看 Consumer 記錄，瞭解啟動程序。

```
gcloud beta run worker-pools logs tail $CONSUMER_WORKER_POOL_NAME --region=$REGION
```

您應該會看到類似下列內容的記錄：

Done job-100

步驟 10 · 約 0 分鐘

10. 疑難排解

首先，請判斷問題是出在 CREMA 服務設定，還是 PubSub 消費者設定。

將 PubSub 消費者自動調度器設為 1，而非 0。如果系統立即開始處理 Pub/Sub 訊息，表示 CREMA 有問題。如果無法處理 Pub/Sub 訊息，表示 Pub/Sub 消費者有問題。

步驟 11 · 約 0 分鐘

11. 恭喜！

恭喜您完成本程式碼研究室！

建議參閱 [Cloud Run](#) 說明文件。

涵蓋內容

- 如何建立 Pub/Sub 主題和訂閱項目，並將訊息推送至該主題。
 - 如何部署 Cloud Run worker 集區 (消費者)，從 Pub/Sub 取用訊息。
 - 如何將 GitHub 上的 CREMA 專案部署為 Cloud Run 服務，根據 Pub/Sub 訂閱中的訊息數量自動調整工作人員集區。
 - 如何在本機執行 Python 指令碼來產生負載，藉此測試自動調度設定。
-

12. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本教學課程所用資源的費用，請刪除您在本程式碼研究室中建立的資源，或刪除整個專案。

刪除本 Codelab 中使用的資源

1. 刪除 Cloud Run CREMA 服務

```
gcloud run services delete $CREMA_SERVICE_NAME --region=$REGION --quiet
```

2. 刪除 Cloud Run worker 集區消費者

```
gcloud beta run worker-pools delete $CONSUMER_WORKER_POOL_NAME --region=$REGION --quiet
```

3. 刪除 Pub/Sub 訂閱項目和主題

```
gcloud pubsub subscriptions delete $SUBSCRIPTION_ID --quiet
gcloud pubsub topics delete $TOPIC_ID --quiet
```

4. 刪除 Parameter Manager 設定

刪除參數中的版本

```
gcloud parametermanager parameters versions delete $PARAMETER_VERSION \
  --parameter=$PARAMETER_ID \
  --location=$PARAMETER_REGION \
  --quiet
```

現在刪除空白參數

```
gcloud parametermanager parameters delete $PARAMETER_ID \
  --location=$PARAMETER_REGION \
  --quiet
```

5. 刪除服務帳戶

```
gcloud iam service-accounts delete "$CREMA_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" --quiet
gcloud iam service-accounts delete "$CONSUMER_SA_NAME@$PROJECT_ID.iam.gserviceaccount.com" --quiet
```

或是刪除整個專案

如要刪除整個專案，請前往「管理資源」，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
